Identifying the LLM Behind the Text: CNN and LSTM Classifiers for LLM Fingerprinting

Team leader: Siddhant Bhagat (solo)

Repository: https://github.com/Sid-Bhagat/llm-fingerprinting

1. Introduction

The goal of this project is to predict which LLM family wrote a given response. A lot of the text people read online is now generated by models like GPT, Claude, PaLM and Llama, so it is worth asking whether each of these models leaves a consistent stylistic fingerprint that a classifier can learn to recognize.

I set this up as a four class text classification problem over GPT, Claude, Llama and PaLM. Each example is a triple of (LLM name, user prompt, LLM response), and I trained the two required architectures on it, a convolutional neural network (TextCNN) and a recurrent neural network (bidirectional LSTM).

This report covers all four research questions, including the two optional ones:

- **RQ1:** Can the LLM family be identified from the response text alone? (Section 4.1)
- **RQ2:** Does the user's prompt help? I compare classifiers trained on the prompt only, the response only, and both together. (Section 4.2)
- **RQ3 (extra credit):** Does the fingerprint generalize to a task domain the classifier never saw during training? (Section 5.3)
- **RQ4 (extra credit):** Which linguistic and structural characteristics separate the families, and how much does removing them hurt performance? (Section 5.2)

Section 5 also compares the neural models against TF-IDF and handcrafted feature baselines, and breaks the errors down by the specific model behind each response.

2. Dataset

The final dataset has 9,192 responses, balanced at about 2,300 per family, taken from the public Chatbot Arena conversations dataset. I picked this dataset because every row already includes a model name, a user prompt and a model response, and because each prompt is answered by two randomly assigned models, which turned out to matter a lot for RQ2.

2.1 Data Source

I used lmsys/chatbot_arena_conversations from Hugging Face (arXiv:2306.05685). It is a gated dataset released under a CC license, and I was granted access after accepting its terms. The raw data has 33,000 rows covering 20 models, and each row is one user prompt answered by two different models in a blind comparison.

2.2 Family Mapping

I grouped the models by the organization or base model behind them, which gave four families. Every model outside these four was dropped, since each one comes from a different base model (RWKV-4-Raven-14B, chatglm-6b, stablelm-tuned-alpha-7b, dolly-v2-12b, oasst-pythia-12b, mpt-7b-chat, fastchat-t5-3b).

Family	Models included
GPT	gpt-4, gpt-3.5-turbo
Claude	claude-v1, claude-instant-v1
Llama	vicuna-13b, vicuna-7b, koala-13b, alpaca-13b, wizardlm-13b, gpt4all-13b-snoozy, guanaco-33b
PaLM	palm-2

One thing to note is that the Llama class is made up of community fine tunes of the original LLaMA, not Meta's own chat model. Several of these, including Vicuna, were fine tuned on ChatGPT conversations, so I expected some confusion between GPT and Llama from the start.

2.3 Preprocessing

1. Kept only single turn English conversations, which left 25,938 rows (51,876 responses).
2. Split each row into two examples, one per model, using the user turn as the input and the assistant turn as the output.
3. Mapped models to families, dropped the other models, and removed empty outputs and exact duplicate (input, output) pairs. This left GPT 6,944, Claude 5,142, Llama 16,895 and PaLM 2,305.
4. Removed self identification. Some models name themselves ("I am Vicuna, trained by LMSYS"), which would let a classifier cheat. I dropped any response that mentions a model or company name such as Claude, Anthropic, OpenAI, ChatGPT, Vicuna, LMSYS,

PaLM, Bard or Google, or that contains the phrase "trained by". I chose to drop these responses instead of masking the names, because a placeholder token would become a signal of its own if one family self identifies more often than the others.

5. Labeled each prompt's domain as code, math or general using keyword rules.
6. Split the data 70/15/15 by question ID, so the same prompt never shows up in two splits.
7. Balanced each split by downsampling every family to the size of that split's smallest family.

The self identification filter removed very few responses, so it was not a major source of leakage in this data:

Family	Responses dropped for self identification
GPT	22
Claude	28
Llama	35
PaLM	7

I split before balancing on purpose. Balancing first and then splitting by question ID can leave the splits unbalanced, while splitting first lets every split be balanced exactly. The final splits are 6,332 training, 1,416 validation and 1,444 test examples, and no question ID is shared between them.

2.4 Domain Labels

The domain labels come from simple keyword rules applied to the prompt. Code is checked first, based on code fences and code keywords. A prompt is labeled math if it contains clear math terms (equation, integral, prime, divisible, solve for, percent), an arithmetic expression such as 2+2 (with dates removed first), a "how many" or "how much" question with at least two numbers, or if it is mostly digits and operators. Everything else is labeled general.

Family	Code	General	Math
GPT	291	1,847	160
Claude	290	1,848	160

Family	Code	General	Math
Llama	298	1,841	159
PaLM	321	1,794	183

2.5 Dataset Limitations

- PaLM is the smallest family, so it sets the class size and limits the dataset to about 9,200 examples.
- The domain labels are heuristic. When I spot checked 20 prompts labeled math, about 16 were real math problems, and there were false positives such as "Compare 9/11 and January 6th."
- All of the models are from 2023, so the results may not carry over to current models.

3. Methodology

I trained a TextCNN and a bidirectional LSTM in PyTorch. Both use the same tokenizer, vocabulary, embedding size and training procedure, so any difference in results should come from the architectures themselves.

3.1 Tokenization

Text is split into words and individual punctuation marks, and I kept case, punctuation and line breaks, with each newline mapped to a `<nl>` token. Most text classification pipelines lowercase everything and strip punctuation, but here formatting is part of the signal I wanted to capture, since models differ in how they use lists, line breaks and headers. The vocabulary is built from the training split only, capped at the 30,000 most frequent tokens, and saved once so that every run uses the same one.

The sequence length depends on the input setting. Input only uses the first 64 prompt tokens, output only uses the first 256 response tokens, and the combined setting uses 64 prompt tokens, a `<sep>` token and 192 response tokens.

3.2 Model Architectures

	TextCNN	BiLSTM
Embedding	128 dimensions, learned	128 dimensions, learned

	TextCNN	BiLSTM
Core layer	Convolutions with filter widths 3, 4 and 5, 100 filters each	One bidirectional LSTM layer, hidden size 128 per direction
Pooling	Max over time for each filter	Max over time across hidden states
Dropout	0.5	0.5
Output	Linear layer to 4 classes	Linear layer to 4 classes

The TextCNN slides filters of width 3 to 5 across the token sequence, so each filter learns to respond to a short phrase, and max pooling keeps the strongest match from anywhere in the text. The BiLSTM reads the sequence in both directions and pools over all of its hidden states, which in theory lets it pick up longer range structure. In both models, padding positions are excluded from the max pool, so the amount of padding in a batch cannot change a prediction. The TextCNN has 3,995,104 trainable parameters and the BiLSTM has 4,105,220. In both cases 3,840,000 of these (about 96%) belong to the embedding table, so the real difference is in the much smaller core layers, with 155,104 parameters for the convolutions and classifier compared to 265,220 for the LSTM and classifier.

3.3 Training Setup

- **Optimizer:** Adam with a learning rate of 0.001, batch size 64 and cross entropy loss.
- **Stopping:** up to 10 epochs, with early stopping on validation macro F1 and a patience of 2. The checkpoint from the best validation epoch is the one evaluated on the test set.
- **Seeds:** every configuration is trained with seeds 0, 1 and 2, and I report the mean and standard deviation.
- **Metric:** macro F1 is the main metric, with accuracy reported alongside it. Since the classes are balanced the two end up close, and chance level for both is 0.25.
- **Hardware:** everything ran on a laptop CPU. The full grid of 18 runs (2 models, 3 input settings, 3 seeds) took about an hour, and most of that time went to the LSTM on full responses.

Before running the full grid, I ran a one epoch smoke test on 20% of the training data for every configuration to make sure the loss was going down and nothing crashed. This also gave me a runtime estimate, which is how I decided a GPU was not necessary.

4. Results

From the response alone, both models identify the LLM family at roughly 2.7 times the chance rate, with the TextCNN reaching 0.688 macro F1. The prompt alone, on the other hand, carries no usable signal.

Input setting	TextCNN macro F1	TextCNN accuracy	BiLSTM macro F1	BiLSTM accuracy
Output only	**0.688 ± 0.005**	0.685 ± 0.006	0.646 ± 0.060	0.643 ± 0.058
Input + output	0.675 ± 0.012	0.672 ± 0.013	0.648 ± 0.010	0.650 ± 0.007
Input only	0.244 ± 0.012	0.261 ± 0.008	0.252 ± 0.009	0.259 ± 0.005

Values are the mean ± standard deviation over seeds 0, 1 and 2 on the 1,444 response test set. Chance level is 0.25. Figure 1 shows the same results as a bar chart.

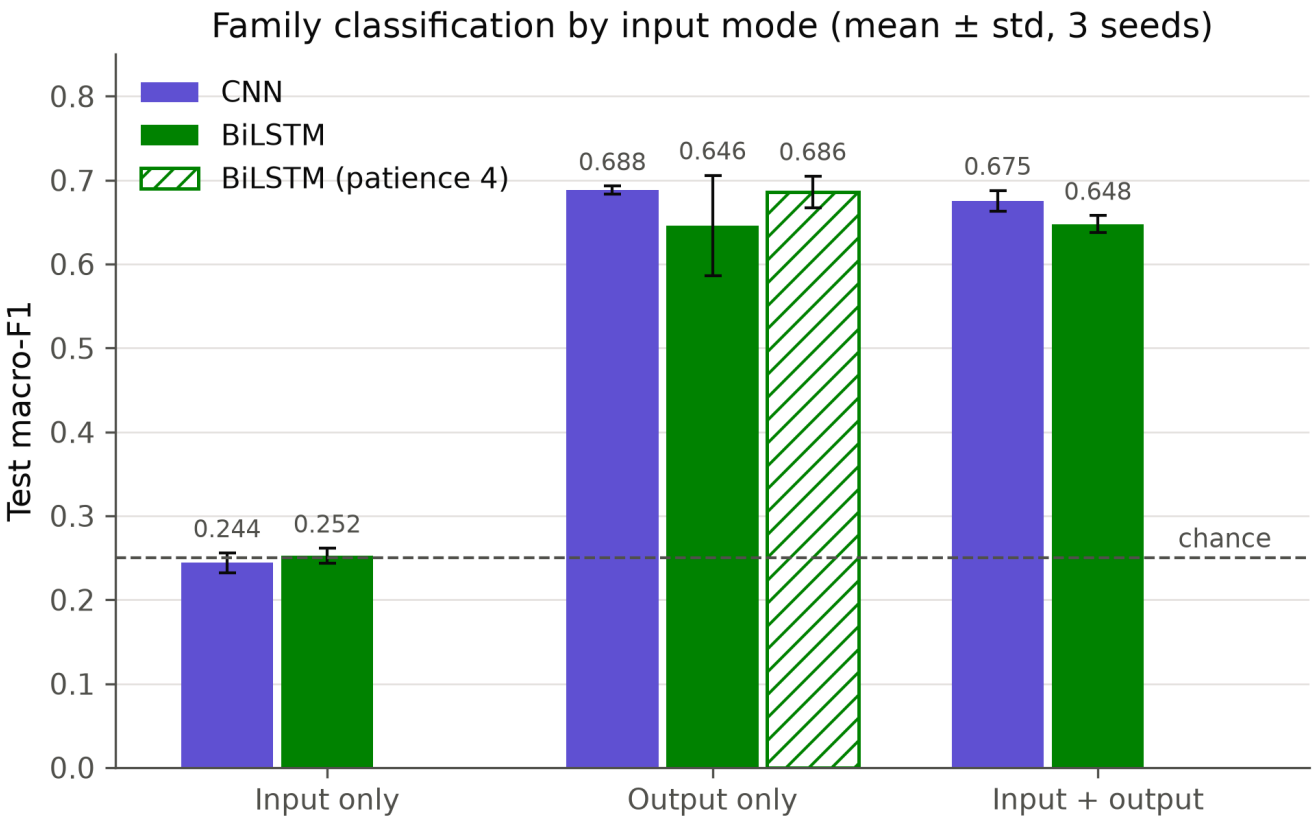


Figure 1. Test macro F1 by input setting, mean ± standard deviation over three seeds. The dashed line marks chance (0.25).

4.1 RQ1: Identifying the Source Model

The answer to RQ1 is yes. Using only the response, the TextCNN reaches 0.688 macro F1 and the BiLSTM reaches 0.646, which are both well above the 0.25 chance level.

The TextCNN is the more reliable of the two, but the BiLSTM's lower average mostly comes from one bad training run rather than a weaker architecture. Its three seeds scored 0.664, 0.694 and 0.579 macro F1. Seeds 0 and 1 kept improving until epochs 8 and 7, and seed 1 actually scored higher than the TextCNN's mean. Seed 2, however, peaked at epoch 2, dipped at epoch 3, and was stopped at epoch 4 before it had a chance to recover, because with a patience of 2 one noisy epoch is enough to end training.

To check this explanation, I reran the three BiLSTM seeds with a patience of 4. Seed 0 gave an identical result, since it had already run the full 10 epochs. Seed 1 trained one epoch longer and finished almost the same, going from 0.694 to 0.693. Seed 2 kept training and improved from 0.579 to 0.700, which ended up being the best of all six BiLSTM runs. The early epochs of every rerun matched the original runs exactly, which also confirms that training is reproducible for a given seed. Overall, the BiLSTM mean went from 0.646 ± 0.060 to 0.686 ± 0.019, which is essentially tied with the TextCNN's 0.688 ± 0.005. I kept patience 2 as the main result, since that is the setting I chose before seeing any results, and I report patience 4 as a sensitivity check. My conclusion is that the two architectures reach about the same accuracy, but the TextCNN gets there more consistently and is much less sensitive to the stopping rule. Figure 2 shows the training loss and validation curves for seed 2 under both settings, which are identical until patience 2 stops the run.

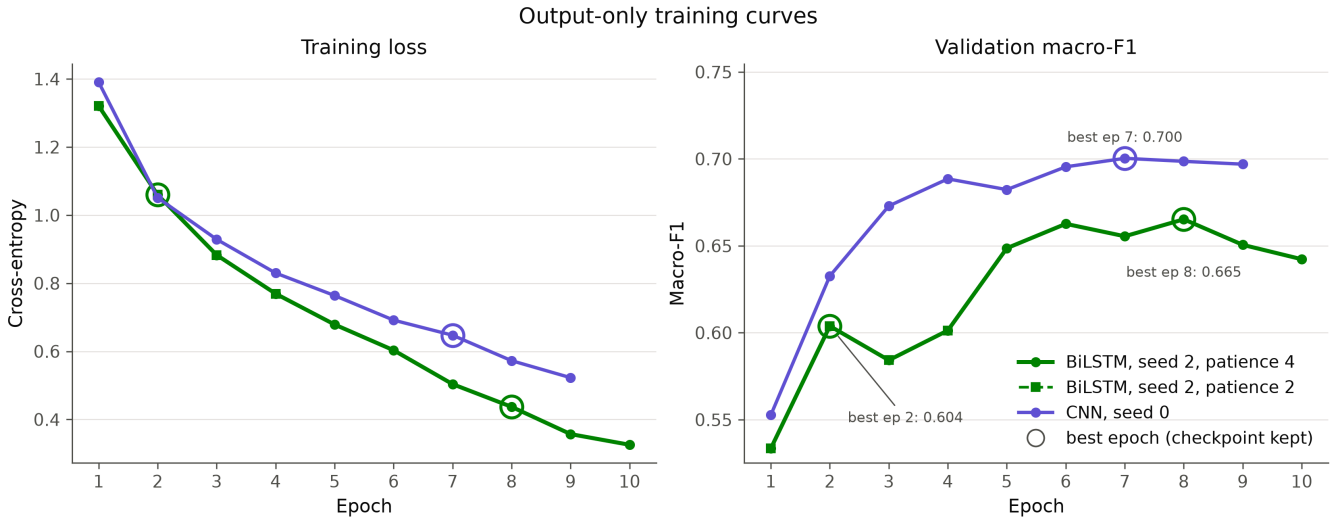


Figure 2. Training loss and validation macro F1 per epoch for the TextCNN (seed 0) and the BiLSTM (seed 2) with patience 2 and 4. Rings mark the best epoch, whose checkpoint is kept. The two BiLSTM runs are identical until patience 2 stops at epoch 4.

The TextCNN, on the other hand, converges quickly and consistently, with a standard deviation of only 0.005 across seeds. My explanation for this is that the strongest cues are local. Opening phrases, common stock phrases and formatting tokens are all short n grams, which a CNN's width 3 to 5 filters can detect directly, while the LSTM has to learn to carry that information across 256 steps. That takes more epochs and makes its training noisier on only about 6,300 examples. I test the idea that the cues are local in the RQ4 analysis.

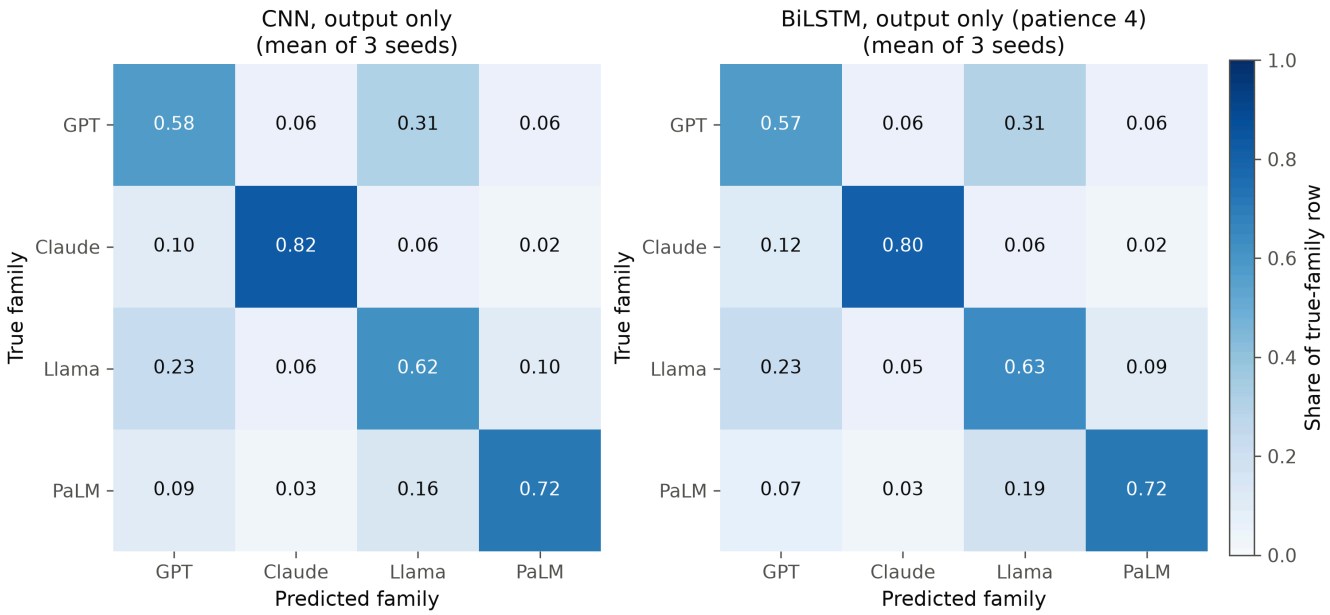


Figure 3. Row normalized confusion matrices on the test set, averaged over three seeds, for the TextCNN (left) and the BiLSTM with patience 4 (right). Each row shows how responses from one true family were classified.

The confusion matrices in Figure 3 show that the families are not all equally easy to identify. Averaged over the three seeds, TextCNN recall is 0.83 for Claude, 0.72 for PaLM, 0.62 for Llama and only 0.58 for GPT. The most common error is GPT being labeled as Llama, which happens for 31% of GPT responses, and the reverse is the second most common, with 23% of Llama responses labeled as GPT. This is what I expected given the data, since several of the Llama fine tunes were trained on text generated by OpenAI models (for example, Alpaca on text-davinci-003 outputs and Vicuna on shared ChatGPT conversations) and learned to imitate that style. Basically, the Llama class contains a lot of secondhand GPT writing, so the line between the two is blurry. Section 5.4 tests this explanation model by model. The BiLSTM shows almost exactly the same pattern, with GPT recall of 0.57, 31% of GPT responses labeled as Llama and 23% of Llama responses labeled as GPT. Since two different architectures make the same mistakes, the confusion most likely comes from the data and not from either model.

4.2 RQ2: Effect of the User Prompt

Prompt only classification

The prompt alone cannot predict the LLM. Both models score at chance from the prompt alone, with 0.244 macro F1 for the TextCNN and 0.252 for the BiLSTM. During these runs the training loss kept going down, from about 1.40 to 1.20, while validation F1 stayed around 0.26. In other words, the models were memorizing individual training prompts, and that did not carry over to new prompts, so early stopping kept an early epoch.

The reason input only fails here comes down to how the data was collected. In Chatbot Arena, a user writes a prompt and two models are assigned to answer it at random, so the prompt has nothing to do with which model replies. And since the prompts are split by question ID, nothing in a test prompt can point to a family. In a dataset where people choose a model for a task, for example sending coding questions to GPT 4, input only classification could beat chance by learning the task distribution. That would not be a real fingerprint, though, so a strong input only result should really be read as a sign of selection bias in the data.

Combining the prompt and the response

Adding the prompt does not improve classification. It lowered TextCNN macro F1 slightly, from 0.688 to 0.675, and left the BiLSTM basically unchanged at 0.648 compared with 0.646. However, there is a confound in this comparison. In the combined setting the response is cut to 192 tokens to make room for the prompt, while the output only setting uses 256 tokens, so the drop could come from seeing less of the response rather than from the prompt itself.

To separate these two effects, I trained the TextCNN on output only with responses cut to 192 tokens, which scored 0.681 ± 0.010. This means about half of the 0.013 drop comes from the shorter response, and the remaining 0.006 that could be blamed on the prompt is smaller than the standard deviation across seeds. So the prompt neither helps nor really hurts, which makes sense when it carries no information about the family.

5. Additional Analyses

5.1 Baseline Comparison

A linear model on TF-IDF n grams performs as well as both neural models. To see how much the CNN and LSTM actually add over simpler methods, I trained logistic regression on TF-IDF features of the response text, once with word unigrams and bigrams and once with character 3 to 5 grams (50,000 features each, case kept, regularization tuned on validation).

Model	Test macro F1
TF-IDF character 3 to 5 grams + logistic regression	0.692
TF-IDF word 1 to 2 grams + logistic regression	0.691
TextCNN	0.688 ± 0.005
BiLSTM, patience 4	0.686 ± 0.019
BiLSTM, patience 2	0.646 ± 0.060
12 handcrafted features + logistic regression	0.435

The TF-IDF models are deterministic, so each is a single run without a standard deviation.

The two TF-IDF variants, the TextCNN and the patience 4 BiLSTM all land within 0.006 of each other, and their per family results are very similar too. The character model gets F1 scores of 0.83 for Claude, 0.76 for PaLM, 0.59 for Llama and 0.58 for GPT, and again GPT and Llama are confused with each other more than any other pair. With about 6,300 training examples, the neural models are not finding anything that a bag of short word or character sequences cannot. This lines up with the rest of my results. If the fingerprint is made of short surface patterns, then a CNN's narrow filters end up learning basically the same features as an n gram model, and the LSTM's ability to model longer range structure does not have anything extra to work with.

Looking at the highest weighted character n grams for each family pointed me to a cue I had not measured directly, which is the character each family uses for bullet points:

Family	Top character n grams
GPT	a newline followed by "- " (hyphen bullets), "As "

Family	Top character n grams
Claude	" • " (a Unicode bullet character), "Here "
PaLM	a newline followed by "* " (asterisk bullets), "**" (bold)
Llama	code fences (```), " and "

Each family formats lists with a different bullet character, which a classifier can pick up from just three characters of text, and the opening words "As" and "Here" match the opening phrases found in RQ4. The bullet character is an especially strong cue for Claude. 540 of Claude's 2,298 responses (23.5%) contain •, compared with 2 or fewer for each of the other families. The markdown stripping ablation in Section 5.2.3 only removes bullets that start a line and are followed by a space, so 37 of those 540 Claude responses (about 7%, and 4 of 80 in the test set) still had a • after stripping. This means the ablation removed most of this cue but not all of it, and its 0.036 drop slightly understates how much formatting contributes.

5.2 RQ4: Distinguishing Characteristics

The fingerprint is both structural and linguistic, but most of it is in the wording, especially in how a response begins. Formatting and length make up a real but smaller share, and the signal overlaps enough that removing any one part only costs between 4 and 7 points of macro F1.

5.2.1 Structural and Length Features

I computed 12 handcrafted features for all 9,192 responses and ranked them using a Kruskal Wallis test of how strongly each one differs across families. Bold use, newline rate, list use and sentence count differ the most, while header use and question marks barely differ at all.

Feature (mean)	GPT	Claude	Llama	PaLM
Words (median)	92	186	102	169
Type token ratio, first 100 words	0.718	0.684	0.669	0.642
Words per sentence	14.2	11.2	16.3	13.4
Newlines per 100 words	6.3	9.7	5.6	7.4

Feature (mean)	GPT	Claude	Llama	PaLM
Share of lines that are list items	0.18	0.36	0.16	0.21
Responses with any bold text	1.2% or less	1.2% or less	1.2% or less	13.3%

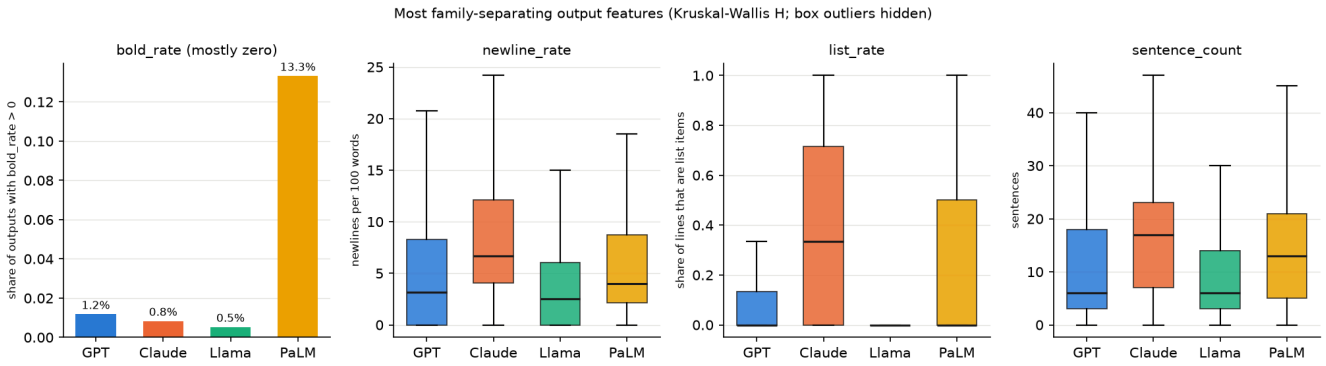


Figure 4. The four features that differ most across families by Kruskal Wallis H. Bold use is shown as the share of responses with any bold text, since most responses have none.

Each family has a recognizable structural habit. Claude writes the longest and most list heavy responses, with the shortest sentences. PaLM is the only family that regularly uses bold text, in 13.3% of its responses compared with about 1% or less for the others. GPT has the highest lexical diversity and the shortest typical response, while Llama writes the longest sentences. GPT and Llama are the closest pair on almost every feature, which matches the confusion between them in RQ1.

A logistic regression trained on just these 12 features reaches 0.435 macro F1. That is well above the 0.25 chance level but far below the TextCNN's 0.688, and its weakest class is again GPT, with an F1 of 0.304. So structure and length explain part of the fingerprint, and the rest of the gap has to come from the words themselves.

5.2.2 Opening Phrases

Family	Most common opening phrases (share of responses)
Claude	"Here is a" (13.5%), "Here are some" (9.2%), "I apologize, but" (5.7%)
PaLM	"I'm not able" (5.0%), "Sure, here is" (4.7%)
GPT	"As an AI" (3.6%), "I'm sorry, but" (2.4%)
Llama	"I'm sorry, but" (2.6%), "As an AI" (1.9%)

Openings turned out to be the clearest single signal. Claude starts more than one in eight responses with "Here is a", and when it refuses it starts with "I apologize" instead of "I'm sorry". PaLM refuses with "I'm not able". GPT and Llama share the same top openings, "As an AI" and "I'm sorry, but", which again suggests the Llama models picked up ChatGPT's phrasing. These phrases do not name a model, so they are not the kind of leakage my filter was meant to remove. They are habitual phrasing, which is exactly the fingerprint this project is looking for. Since these cues are three word phrases near the start of the text, they also support my explanation from RQ1 that the TextCNN's short filters are a good fit for this task.

5.2.3 Ablation Experiments

To test how much each kind of signal matters, I retrained both models on output only text with formatting removed (code fences, headers, bullets, bold and italic markers, and repeated blank lines) and, separately, with each response cut to its first 100 tokens.

Setting	TextCNN macro F1	BiLSTM macro F1
Full response, 256 tokens	0.688 ± 0.005	0.646 ± 0.060
Markdown stripped	0.652 ± 0.018	0.650 ± 0.016
First 100 tokens only	0.640 ± 0.032	0.629 ± 0.007

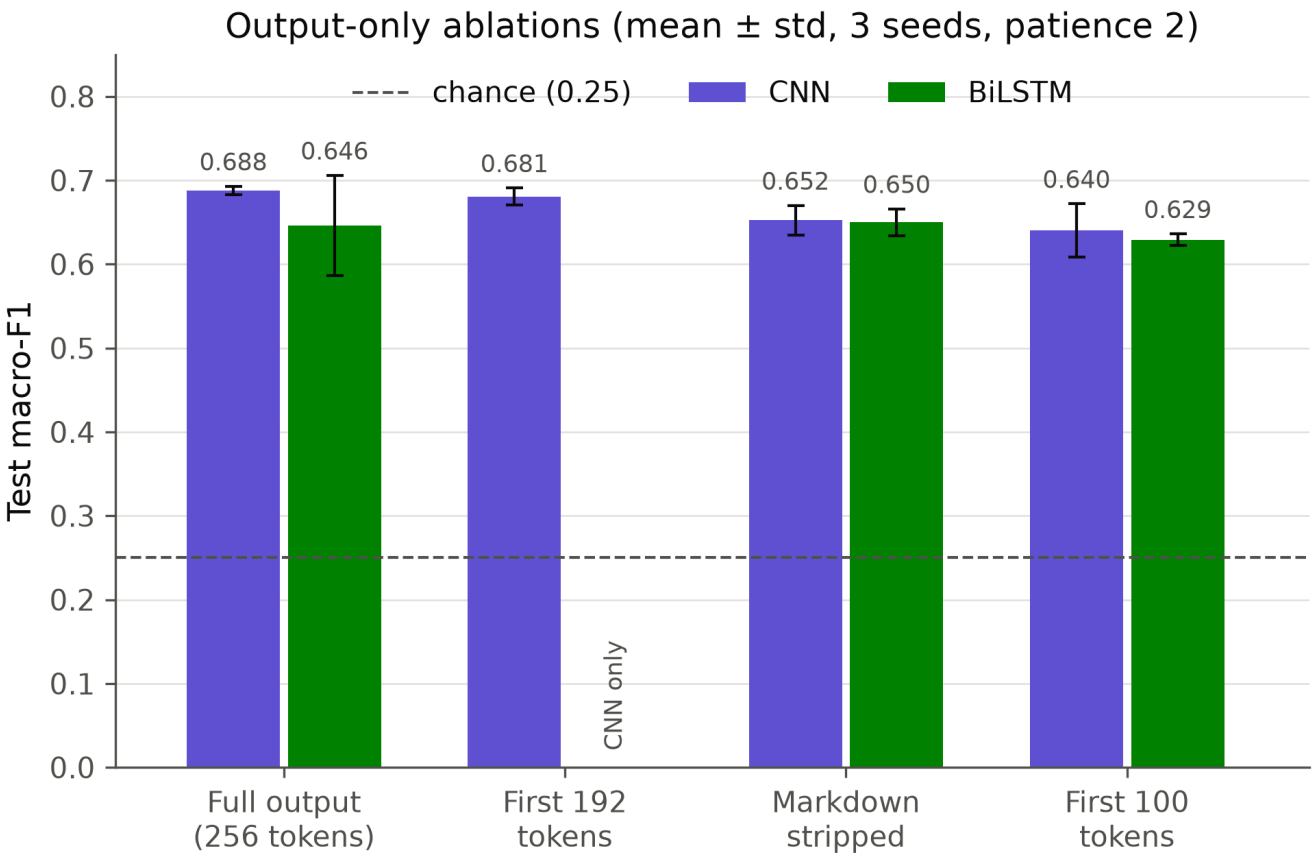


Figure 5. Output only ablations, mean ± standard deviation over three seeds, all with patience 2. The 192 token control was run for the TextCNN only.

Figure 5 shows these results as a chart. Stripping markdown lowers the TextCNN by 0.036, so formatting does contribute, but the model still reaches 0.652 without it. This is interesting when compared with the feature model, because the structural features alone are worth 0.435, yet removing them from the text costs less than 0.04. That means the network can find most of the same information in the wording. Cutting responses to 100 tokens lowers the TextCNN by 0.048, which also removes most of the length signal, since almost every response longer than 100 tokens now looks the same. Even so, the first 100 tokens alone still give 0.640, so the fingerprint is concentrated near the start of a response.

To test the role of openings directly, I also retrained the TextCNN with the first sentence of every response removed. This caused the largest drop of any ablation, from 0.688 to 0.622 ± 0.021, even though it removes much less text than truncation does. About 8% of test responses (116 of 1,444) were left unchanged because they had no sentence boundary or nothing after the first sentence, so if anything the real effect of the opening is slightly larger. In other words, a single opening sentence carries more identifying signal

than all of the markdown formatting in a response. The BiLSTM comparisons are noisier, because its patience 2 baseline includes the early stopped seed from RQ1, but its scores under both ablations fall in the same range.

To answer the RQ4 questions directly, the classifier seems to rely mostly on wording, especially opening and refusal phrases, with formatting and length as secondary cues. The signals are both linguistic and structural, and they overlap a lot. Removing the opening sentence hurts the most (a drop of 0.066), more than removing formatting (0.036) or everything after the first 100 tokens (0.048). Even so, no single removal brings performance anywhere close to chance, so no one signal is responsible for the fingerprint on its own.

5.3 RQ3: Cross Domain Generalization

The fingerprint partly generalizes. Even when trained without any code prompts, both models still identify the family behind code answers at more than twice chance, but they lose about 13 points of macro F1 compared with the domains they were trained on.

5.3.1 Experimental Setup

I trained both models on output only text from general and math prompts, then tested them in two ways. The in domain test uses the general and math prompts from the regular test split (1,249 responses). The cross domain test uses every code prompt in the dataset, pooled from all three splits and balanced to 290 responses per family (1,160 responses). Pooling is safe here because no code example is used for training or early stopping in these runs, and it gives a test set about six times larger than the code portion of the test split alone. Both test conditions use the same trained models, so any difference comes from the test domain and not from the amount of training data. The TextCNN used the default patience of 2, and the BiLSTM used a patience of 4, since RQ1 showed that patience 2 makes it unstable.

5.3.2 Results

Model	Test domain	Macro F1	GPT	Claude	Llama	PaLM
TextCNN	General + math (seen)	0.689 ± 0.024	0.544	0.850	0.600	0.763
TextCNN	Code (unseen)	0.563 ± 0.016	0.507	0.695	0.413	0.636
BiLSTM	General + math (seen)	0.667 ± 0.014	0.573	0.833	0.543	0.717
BiLSTM	Code (unseen)	0.528 ± 0.019	0.510	0.659	0.372	0.571

Per family columns are test F1 averaged over seeds 0, 1 and 2.

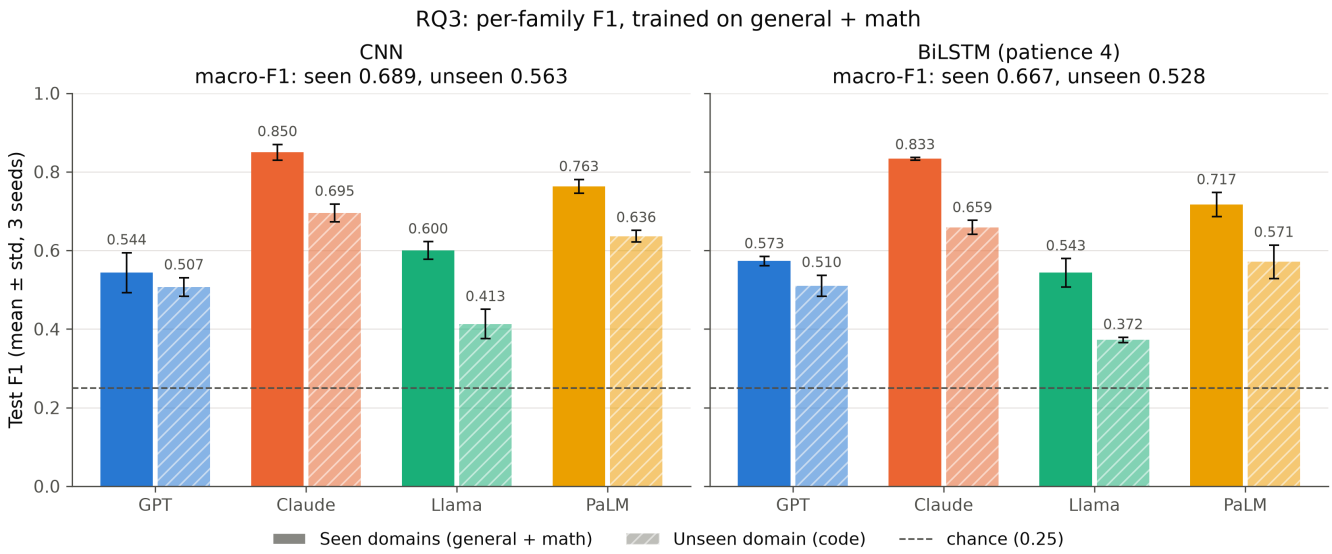


Figure 6. Per family test F1 for models trained on general and math prompts, tested on the same domains (solid) and on unseen code prompts (hatched). Mean ± standard deviation over three seeds.

Performance does drop on the unseen domain, by 0.126 macro F1 for the TextCNN and 0.139 for the BiLSTM. Even so, both models still score 0.53 to 0.56 on code, which is more than twice the 0.25 chance level, so a good part of the fingerprint carries over to a domain the models never saw.

The families behave very differently across domains. For the TextCNN, Llama drops the most, by 0.19, while for the BiLSTM it ties with Claude, with both dropping 0.17. Either way, Llama ends up as the hardest family to identify on code for both models, at 0.41 and 0.37. For the TextCNN this is because it dropped the most, since it actually started above GPT in domain, and for the BiLSTM it is because it started lower. Claude and PaLM each lose 0.13 to 0.17 but stay the two most identifiable families, so their habits seem to survive the change of task. GPT barely changes, losing only 0.04 and 0.06, but in domain it was already one of the two hardest families along with Llama, mostly because of their confusion with each other, so it had less domain specific signal to lose. One possible explanation for Llama's large drop is that these community fine tunes learned a lot of their coding style from ChatGPT outputs, so their code answers look even more like GPT than their general answers do. I did not test this directly.

So is the classifier learning a general fingerprint or task specific patterns? The answer seems to be both. The above chance accuracy on unseen code points to a general fingerprint that does not depend on topic, which fits the RQ4 finding that opening and refusal phrases carry a lot of the signal. At the same time, the 13 point drop shows that

part of what the models learn is tied to how each family writes the kinds of answers seen in training. Code answers are also partly made up of code itself, which leaves less room for the prose habits the classifiers depend on.

5.3.3 Limitations

- The seen and unseen test sets come from different pools of prompts, so the two macro F1 scores are comparable as estimates but were not measured on matched examples. The in domain set is also very slightly unbalanced, with 310 to 318 responses per family.
- The shared vocabulary was built from the full training split, which includes code prompts. This means code specific words have embeddings, but those embeddings were never updated by a code example, so the cross domain models gain little or no advantage from it.
- Code is the only held out domain. Math was too small (about 160 responses per family) to test on its own.

5.4 Error Analysis by Model

To test the explanation from RQ1 for the GPT and Llama confusion, I saved every test prediction from the output only models (TextCNN, and BiLSTM with patience 4) and broke the results down by the specific model behind each response. It turns out the confusion is driven mainly by GPT 3.5, not by any single Llama fine tune.

Model	Family	Test responses	Correct	Labeled GPT	Labeled Llama
gpt-4	GPT	177	65.9%		28.8%
gpt-3.5-turbo	GPT	184	49.5%		33.5%
claude-v1	Claude	234	85.2%	7.7%	5.0%
claude-instant-v1	Claude	127	77.4%	13.1%	7.1%
vicuna-13b	Llama	101	66.7%	20.1%	
vicuna-7b	Llama	48	65.3%	20.8%	
koala-13b	Llama	90	57.8%	24.8%	
alpaca-13b	Llama	77	60.2%	23.8%	
wizardlm-13b*	Llama	16	54.2%	31.2%	
gpt4all-13b-snoozy*	Llama	14	66.7%	19.0%	
guanaco-33b*	Llama	15	57.8%	22.2%	
palm-2	PaLM	361	72.1%	8.8%	16.1%

TextCNN results, averaged over seeds 0, 1 and 2. Blank cells are the model's own family. * Fewer than 30 test responses, so these rows are too small to interpret.

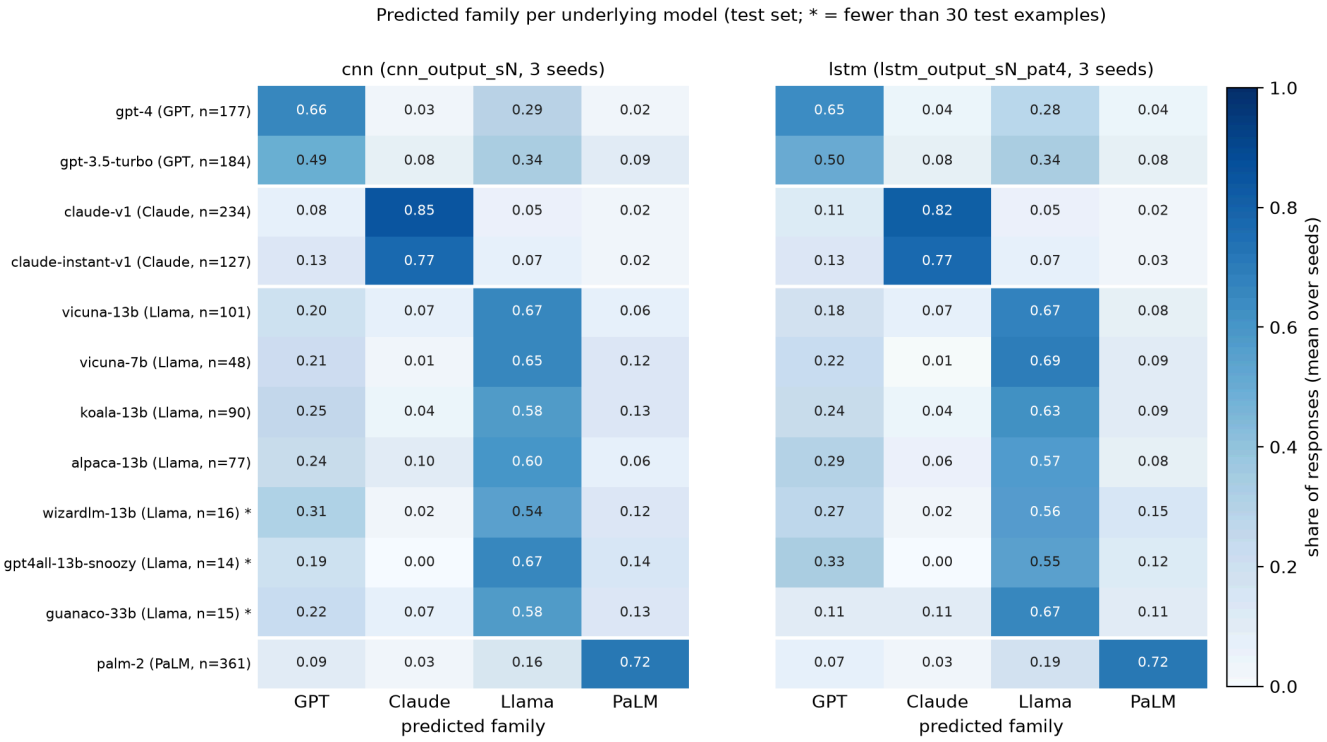


Figure 7. Share of each underlying model's test responses predicted as each family, averaged over three seeds, for the TextCNN (left) and the BiLSTM with patience 4 (right). Asterisks mark models with fewer than 30 test responses.

Vicuna

I originally expected Vicuna, since it was trained directly on ChatGPT conversations, to be the main source of the confusion. However, its responses are labeled GPT slightly less often (20 to 21%) than Koala's or Alpaca's (24 to 25%). The GPT likeness is spread across the whole Llama class instead, which makes sense given that several of these fine tunes were trained on text generated by OpenAI models, not just Vicuna. So my original, more specific explanation was not supported.

GPT 3.5 versus GPT 4

Within the GPT family, GPT 4 is identified correctly 65.9% of the time, but GPT 3.5 only 49.5% of the time, and a third of GPT 3.5 responses are labeled Llama. GPT 3.5 and its close predecessors are the models whose outputs the Llama fine tunes were trained to imitate, so the model being imitated is the one most often mistaken for its imitators. This ended up being stronger evidence for the imitation explanation than the Vicuna comparison would have been.

Claude v1 versus Claude Instant

The smaller model within a family is also harder to identify. Claude Instant is identified less often than Claude v1 (77.4% against 85.2%) and gets labeled GPT almost twice as often.

The BiLSTM shows the same pattern. GPT 3.5 is again much harder than GPT 4 (50.4% against 64.8%), and Vicuna is again labeled GPT no more often than Koala or Alpaca (18 to 22% against 24 to 29%).

6. Lessons Learned

The biggest thing I learned from this project is that the decisions around the models mattered more than the models themselves. The CNN and LSTM ended up tied with a simple logistic regression on TF-IDF features, while choices about leakage, splitting, early stopping and ablation design changed my conclusions several times.

6.1 Data Handling

Looking at the raw data before writing any pipeline code paid off right away. The dataset had 20 models, and my first family map missed several LLaMA fine tunes. I also had to decide what counts as leakage. A response that names its model ("I am Vicuna, trained by LMSYS") gives the answer away, so I dropped those responses instead of masking the names, since a mask token would become a signal of its own. On the other hand, habitual phrases like "As an AI" do not name a model, so I kept them, because that kind of phrasing is exactly the fingerprint the project is about. Splitting by question ID mattered just as much, since without it the same prompt could end up in both training and test and the input only results would mean nothing. I also learned to split first and balance second, because balancing first leaves the splits unbalanced.

6.2 Hyperparameter Sensitivity

With an early stopping patience of 2, the LSTM looked clearly worse and less stable than the CNN. One seed stopped at epoch 4 with 0.579 macro F1, but with a patience of 4 the same seed reached 0.700, the best LSTM result I got. Instead of dropping the bad seed or quietly changing the setting, I kept the original result as the main one and reported the rerun as a sensitivity check. That changed my answer from "the CNN is better" to "the two are about equally accurate, but the CNN is much more robust to training settings."

6.3 Importance of Baselines

The TF-IDF logistic regression scored 0.692, which tied both neural models. If I had not run it, I would have given the CNN credit for learning something more sophisticated than it actually did. Instead, the baseline showed that the fingerprint is made of short surface patterns, and its top features pointed to a cue I had not thought to measure, which is that each family uses a different bullet character.

6.4 Testing Hypotheses

I expected Vicuna, which was trained on ChatGPT conversations, to explain the confusion between GPT and Llama. The per model error analysis showed that Vicuna was no more GPT like than Koala or Alpaca, so that specific idea was wrong. The same analysis found the real pattern instead: GPT 3.5, the model these fine tunes imitated, is the GPT model most often mistaken for Llama. Reporting the hypothesis that failed made the final explanation more convincing, not less.

6.5 Confounding Factors

Adding the prompt seemed to lower accuracy slightly, until I noticed that the combined setting also cut the response from 256 to 192 tokens. A control run showed that about half of the drop came from the shorter response. In RQ3, the seen and unseen test sets came from different pools, which I could not avoid but needed to state. In both cases the first comparison looked clean, and it was only by checking what else differed between the two settings that the result became trustworthy.

6.6 Verifying Claims Across Models

Several of my summary statements, like "GPT is the hardest family" and "Llama drops the most on code," turned out to be true for one model but not the other. They came from reading one table and generalizing from it. Checking each claim against the result files for both models caught these before they made it into the report.

6.7 Ablation Design

My markdown stripping ablation required a space after a bullet, so 7% of Claude's • bullets survived it, and I only noticed because the TF-IDF baseline ranked " • " as Claude's top feature. Similarly, removing the first sentence left 8% of responses unchanged. Neither problem changed my conclusions, but both showed me that an ablation's effect has to be judged by what it actually removed, not by what it was supposed to remove.

6.8 Engineering

- **Resumable and reproducible runs.** Every run script skipped runs that had already finished, so pausing or crashing never cost more than one run. Fixed seeds made reruns identical, which let me verify results and test the patience explanation exactly.
- **Knowing the tools.** Python buffers its output when writing to a file, so a training run that was working fine looked frozen until I checked the process directly.
- **Verifying AI assisted work.** I used Claude to help plan and draft and Claude Code to help write and run the pipeline. This was fast, but it also made mistakes, including a wrong claim in my own notes about a rerun being identical. Checking every number against the result files caught these, and next time I would build that check in from the start.

6.9 Future Work

With more time, I would collect my own data by sending the same prompts to current models through their APIs, which would give matched comparisons across families and a larger, more recent dataset. I would also choose the early stopping settings by checking their stability across seeds before running the full grid, and I would fine tune a pretrained transformer to see whether a contextual model finds signals beyond the surface patterns that the CNN, LSTM and TF-IDF models all rely on.

7. AI Use

Claude AI was used for planning and drafting code.